

Computer Networks

Lecture 31 Transport Layer

Content



- **TCP Congestion Control**

TCP Congestion Control

Congestion Control Variants

Over the years, several variants have been developed to optimize performance under different network conditions.

The following are TCP congestion control variants:

- TCP Tahoe
- TCP Reno
- TCP New Reno
- TCP Vegas

TCP Congestion Control

Congestion Control Variants: TCP Tahoe

Case 1: Loss Detected by Timeout

- $ssthresh = CWND / 2$
- $CWND = 1 \text{ MSS}$
- Retransmit lost segment
- Restart slow start

Case 2: Loss Detected by 3 Duplicate ACKs

- $ssthresh = CWND / 2$
- $CWND = 1 \text{ MSS}$
- Retransmit lost segment
- Restart slow start

TCP Congestion Control

Congestion Control Variants: TCP Reno

Case 1: Loss Detected by Timeout

- $ssthresh = CWND / 2$
- $CWND = 1 \text{ MSS}$
- Retransmit lost segment
- Restart slow start

Case 2: Loss Detected by 3 Duplicate ACKs

- Retransmit lost segment without waiting for timeout
- $ssthresh = CWND / 2$
- $CWND = ssthresh + 3 \text{ MSS}$

When the ACK for the retransmitted segment arrives:

- $CWND = ssthresh$
- Enter Congestion Avoidance

TCP Congestion Control

Congestion Control Variants: TCP NewReno

TCP Reno works well when only one segment is lost. But if multiple segments are lost in the same window, Reno may not recover efficiently.

Sender sends $\Rightarrow$ 1 2 3 4 5 6 7 8

Suppose segments 3 and 6 are lost.

The receiver keeps sending duplicate ACKs for segment 3.

ACK 3

ACK 3

ACK 3

After 3 duplicate ACKs, Reno retransmits segment 3. But after segment 3 is recovered, the receiver still cannot acknowledge all data because segment 6 is also missing. So Reno may exit fast recovery too early and later wait for timeout. That slows down recovery.

TCP Congestion Control

Congestion Control Variants: TCP NewReno

Key Idea

Do not leave fast recovery after recovering only one lost segment. Stay in fast recovery until all data that were outstanding when loss was detected are acknowledged.

Case 2: Loss Detected by 3 Duplicate ACKs

- $ssthresh = CWND / 2$
- Retransmit lost segment without waiting for timeout
- $CWND = ssthresh + 3 \text{ MSS}$

If a partial ACK arrives

- Partial ACK received
- Retransmit next lost segment
- Remain in fast recovery

Exit fast recovery only after full ACK

- $CWND = ssthresh$
- Enter Congestion Avoidance

TCP Congestion Control

Congestion Control Variants: TCP Vegas

TCP Vegas is a congestion control mechanism that tries to detect congestion before packet loss happens.

TCP Vegas compares: Expected throughput
 Actual throughput

If the actual throughput is much lower than expected, Vegas assumes that packets are waiting in router queues. That means congestion may be forming.

TCP Congestion Control

Congestion Control Variants: TCP Vegas

Expected Throughput

Expected throughput is calculated assuming there is no congestion.

$$\text{Expected throughput} = \frac{\text{CWND}}{\text{BaseRTT}}$$

Where

CWND $\Rightarrow$ *Congestion Window*

BaseRTT = *minimum observed RTT*

Actual Throughput

Actual throughput is calculated using the current measured RTT.

$$\text{Actual throughput} = \frac{\text{CWND}}{\text{RTT}}$$

Where

CWND $\Rightarrow$ *Congestion Window*

RTT = *Current Round Trip Time*

TCP Congestion Control

Congestion Control Variants: TCP Vegas

Decision

Usually two thresholds are used:

α = lower threshold

β = upper threshold

If Difference $< \alpha$:
increase CWND

If $\alpha \leq \text{Difference} \leq \beta$:
keep CWND unchanged

If Difference $> \beta$:
decrease CWND

Where

$\text{Difference} = \text{Expected throughput} - \text{Actual throughput}$

Question

In a TCP Vegas connection, the congestion window size is 20 MSS. The minimum observed round-trip time, BaseRTT, is 50 ms, and the current measured RTT is 100 ms. The Vegas threshold values are $\alpha = 2$ and $\beta = 6$.

Using the TCP Vegas congestion control rule, determine whether the sender should increase, decrease, or keep the congestion window unchanged. Also find the new congestion window size.